

HIP Notebook Exercise 题面与题解

Lab1 Vector Addition

Exercise: Thread Index Calculation

题目原文

Fill in the global index for each thread:

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	_____
2	128	50	_____
5	64	0	_____

题解

全局线程编号公式为：

```
globalIdx = blockIdx.x * blockDim.x + threadIdx.x;
```

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

Exercise 1: GPU Specifications

题目原文

Based on the output above, record your GPU's specifications:

Property	Your GPU Value	Significance
Device Name	_____	GPU model identifier
Compute Units	_____	Number of parallel execution units
Wavefront Size	_____	Threads per warp (32 on RDNA , 64 on CDNA)
Max Threads per Block	_____	Upper limit for blockDim
Max Shared Memory per Block	_____	LDS available per block

Question: How many total threads can your GPU execute simultaneously if all CUs are fully utilized?

Your calculation: _____

题解

根据已执行 Notebook 的设备检测输出：

Property	实测值	说明
Device Name	Radeon 8060S Graphics	rocminfo 与 HIP 程序均一致
Compute Units	40 个物理 CU	rocminfo 报告 40 CU；hipDeviceProp_t::multiProcessorCount 报告 20，后者按 RDNA WGP 计数
Wavefront Size	32 threads	RDNA 3.5 默认 wave32
Max Threads per Block	1024 threads	HIP 设备属性输出
Max Shared Memory per Block	65,536 bytes (64 KiB)	hipDeviceProp_t::sharedMemPerBlock 实测值

同时活跃线程数估计 = Compute Units × 每个 CU 可同时驻留的 wavefront 数 × Wavefront Size

补充属性显示：HIP 的一个 multiprocessor 对应一个 WGP，系统共有 20 个 WGP；每个 WGP 最多 2,048 个活跃线程，即 64 个 wave32。等价的物理 CU 视角为 40 CU、每 CU 32 个 wave32。因此同时活跃线程上限为：

$20 \text{ WGP} \times 2048 \text{ threads/WGP} = 40,960 \text{ threads}$

等价计算为 $40 \text{ CU} \times 32 \text{ wavefronts/CU} \times 32 \text{ threads/wavefront} = 40,960 \text{ threads}$ 。该值是设备线程驻留上限，不是一次 kernel 能 launch 的总线程数。

Exercise 2: Implement the Kernel

题目原文

Complete the kernel in `vector_add_kernel.hip` based on the structure above.

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    // Step 1: Calculate global thread index
    int idx = _____; // Fill in

    // Step 2: Boundary check (important when N is not divisible by block size)
    if (_____) { // Fill in the condition
        // Step 3: Perform computation
        C[idx] = _____; // Fill in
    }
}
```

题解

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

Exercise 3: Launch Configuration Calculation

题目原文

Given $N = 1000$ elements, complete the table:

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\text{ceil}(1000/64)$	_____	_____	_____
128	$\text{ceil}(1000/128)$	_____	_____	_____
256	$\text{ceil}(1000/256)$	_____	_____	_____
512	$\text{ceil}(1000/512)$	_____	_____	_____

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer: _____

题解

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\text{ceil}(1000/64)$	16	1024	24
128	$\text{ceil}(1000/128)$	8	1024	24
256	$\text{ceil}(1000/256)$	4	1024	24
512	$\text{ceil}(1000/512)$	2	1024	24

四种 block size 的线程利用率相同，均为 $1000/1024$ 。但真实性能还取决于 wavefront 对齐、occupancy、寄存器占用、访存模式和调度开销，所以线程利用率最高不一定代表运行最快。

Exercise 4: Analyze Your Results

题目原文

Based on the multi-trial results above:

- 1. Which block size achieved the **lowest mean execution time**? _____
- 2. Which block size achieved the **highest memory bandwidth**? _____
- 3. Are the differences between block sizes **statistically significant** (look at error bars)? _____

4. Why does Vector Add performance depend more on bandwidth than on block size?

Your answer: _____

题解

1. 平均 kernel 时间最低的是 block size 128: **0.0210 ms**。
2. 按 **12N/time** 计算的有效带宽最高的也是 block size 128, 约为 **599 GB/s**。
3. 32--1024 threads/block 的误差线大范围重叠, 不能据此认为这些配置之间存在稳定、普遍的显著差异; 16 threads/block 的均值更高且波动较小, 表现出较明确的不足一个 wavefront 的损失。
4. Vector Add 每个元素只做一次加法, 但需要从全局内存读取 **A[i]**、**B[i]** 并写回 **C[i]**, 算术强度很低, 主要受 global memory bandwidth 限制。因此只要 block size 能提供足够并行度并保持访存合并, 性能差异通常更多来自带宽而不是计算能力。

Exercise 5: Wavefront Analysis

题目原文

Question 1: A kernel is launched with block size 100. How many wavefronts does each block require? How many lanes are wasted?

- Wavefronts per block: _____ (hint: $\lceil 100/32 \rceil$)
- Wasted lanes: _____ (hint: $\text{wavefronts} \times 32 - 100$)

Question 2: If the same kernel were compiled with **-mwavefrontsize64**, how would the answers change?

- Wavefronts per block (wave64): _____ (hint: $\lceil 100/64 \rceil$)
- Wasted lanes (wave64): _____

Question 3: What happens when threads within a wavefront take different branches (if-else)?

Your answer: _____

题解

wave32 下：需要 $\text{ceil}(100/32)=4$ 个 wavefront，浪费 $4*32-100=28$ 个 lanes。
wave64 下：需要 $\text{ceil}(100/64)=2$ 个 wavefront，浪费 $2*64-100=28$ 个 lanes。同一 wavefront 内线程走不同分支会发生 branch divergence，硬件需要分路径执行，部分 lanes 暂时 inactive，有效并行度下降。

Exercise 6: Occupancy Calculation

题目原文

For Radeon 8060S (RDNA 3.5): 20 CUs, Warp Size = 32, Max Blocks per CU = 64, Max Threads per Block = 1024

To calculate occupancy, you need to know the max wavefronts per CU (check `rocminfo` output).

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	_____
64	2	_____	_____	_____
128	4	_____	_____	_____
256	8	_____	_____	_____
512	16	_____	_____	_____
1024	32	_____	_____	_____

Note: Use `rocminfo` output from Exercise 1 to determine actual hardware limits for your device.

题解

补充设备属性给出：`maxThreadsPerMultiProcessor=2048`、`warpSize=32`、`maxBlocksPerMultiProcessor=64`，所以每个 HIP multiprocessor (WGP) 最多驻留 64 个 wavefront。Vector Add 不使用动态 LDS；忽略寄存器限制时，计算式为：

```
blocks_that_fit = min(64, floor(64 / wavefronts_per_block))
active_wavefronts = blocks_that_fit * wavefronts_per_block
```

Block Size	Wavefronts/Block	Blocks that fit/WGP	Active Wavefronts	Thread-limit Occupancy
32	1	64	64	100%
64	2	32	64	100%
128	4	16	64	100%
256	8	8	64	100%
512	16	4	64	100%
1024	32	2	64	100%

表中 occupancy 依据设备线程和 block 上限计算。实际 kernel 还可能受寄存器限制；本 Vector Add kernel 不使用 LDS，因而不存在每 block LDS 限制。

Exercise 7: Analyze Sub-Wavefront Results

题目原文

Based on the multi-trial results above:

- Mean time for block size 8 (sub-wave): _____ ms
- Mean time for block size 16 (sub-wave): _____ ms
- Mean time for block size 32 (wave-aligned): _____ ms
- Mean time for block size 256 (large): _____ ms

Question 1: Is there a noticeable performance penalty for sub-wavefront block sizes (8, 16) compared to wave-aligned sizes (32, 64, 128)?

Your answer: _____

Question 2: On this RDNA 3.5 GPU, the wavefront size is 32. If you were on an MI100 (wavefront = 64), which block sizes would become sub-wavefront?

Your answer: _____

题解

实测均值为：block size 8=0.0548 ms，16=0.0446 ms，32=0.0188 ms，256=0.0198 ms。

8 和 16 的均值分别约为 block size 32 的 $2.91\times$ 和 $2.37\times$ ，存在明显 sub-wavefront 损失。block size 16 的标准差为 0.0326 ms ，波动较大，但其均值仍明显高于 32、64、128 和 256。若换成 MI100 的 wave64，则小于 64 的 block size 都是 sub-wavefront；非 64 整数倍的 block size 也会在最后一个 wavefront 浪费 lanes。

Challenge Exercise

题目原文

Modify the kernel to process **multiple elements per thread**.

```
__global__ void vector_add_multi(const float* A, const float* B, float* C,
                                int N, int elements_per_thread) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = _____; // Fill in: total number of
threads

    for (int i = tid; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

Question: What is the advantage of this approach over the 1:1 thread-to-element mapping?

题解

```
stride = blockDim.x * gridDim.x;
```

优点：线程总数固定时仍能覆盖任意大的数组，每个线程处理多个元素，减少 kernel 启动与调度开销对每个元素的摊销成本，也便于处理超大 N。

Lab2 Matrix Transpose

Exercise 1: 2D Indexing

题目原文

Given `blockDim = (32, 32)`, calculate the global indices:

blockIdx	threadIdx	Global (idx, idy)
(0, 0)	(5, 10)	(_____, _____)
(1, 0)	(0, 0)	(_____, _____)
(2, 3)	(15, 20)	(_____, _____)

Question: For a matrix with **rows=100**, **cols=200**, how many blocks are needed?

Your calculation: _____

题解

二维全局坐标：

```

idx = blockIdx.x * blockDim.x + threadIdx.x;
idy = blockIdx.y * blockDim.y + threadIdx.y;

```

blockIdx	threadIdx	Global (idx, idy)
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(32, 0)
(2, 3)	(15, 20)	(79, 116)

矩阵 **rows=100**, **cols=200**, block 为 **32x32**, 需要 **ceil(200/32) * ceil(100/32)**
 = **7 * 4** 个 blocks。

Exercise 2: Understand the Index Transformation

题目原文

For a 4×6 matrix (rows=4, cols=6), fill in the table:

Thread (idx, idy)	Input Index	Output Index
(0, 0)	0*6+0 = 0	0*4+0 = 0
(1, 0)	_____	_____
(0, 1)	_____	_____

Thread (idx, idy)	Input Index	Output Index
(3, 2)	_____	_____

Question: Why is the write index formula $\text{idx} * \text{rows} + \text{idy}$ instead of $\text{idx} * \text{cols} + \text{idy}$?

Your answer: _____

题解

Thread (idx, idy)	Input Index	Output Index
(0, 0)	$0*6+0 = 0$	$0*4+0 = 0$
(1, 0)	$0*6+1 = 1$	$1*4+0 = 4$
(0, 1)	$1*6+0 = 6$	$0*4+1 = 1$
(3, 2)	$2*6+3 = 15$	$3*4+2 = 14$

转置后输出矩阵形状为 $\text{cols} \times \text{rows}$ ，输出矩阵每行长度是原矩阵的 rows ，所以写下标是 $\text{idx} * \text{rows} + \text{idy}$ 。

Exercise 3: Performance Analysis

题目原文

Record execution times:

Test	Dimensions	Elements	Time
1	16×16	256	_____
2	128×32	4,096	_____
3	1×1024	1,024	_____
4	1001×2001	2,003,001	_____

Question: The large test has ~500x more elements than test 2. Is the time 500x longer? Why or why not?

Your answer: _____

题解

十次 HIP event 测量的平均时间为：

Test	Dimensions	Elements	Mean Time	Std
1	16×16	256	0.0062 ms	0.0026 ms
2	128×32	4,096	0.0085 ms	0.0020 ms
3	1×1024	1,024	0.0064 ms	0.0021 ms
4	1001×2001	2,003,001	0.1119 ms	0.0019 ms

大测试约为测试 2 的 489 倍元素数，但时间只增长约 $0.1119/0.0085=13.16\times$ 。小规模任务由固定 launch 开销和并行度不足主导；大规模任务能够摊薄固定成本。当前实现为直接转置，连续读、跨步写，其访存模式也影响比例。

Exercise 4: Coalescing Analysis

题目原文

For a 1024×1024 matrix with 32×32 thread blocks (one wave32 = one row of the block):

Question 1: When reading `input[idy * cols + idx]`, are consecutive threads (in `threadIdx.x`) reading consecutive addresses?

Your answer: _____

Question 2: When writing `output[idx * rows + idy]`, what is the stride between addresses written by consecutive threads?

Your answer: _____

Question 3: If each memory transaction fetches 128 bytes (32 floats), how many transactions are needed for 32 threads (one wavefront) to write non-coalesced?

Your answer: _____

题解

1. 是。连续 `threadIdx.x` 对应连续 `idx`，读取地址连续。

2. stride 是 `rows` 个元素；本题 `rows=1024`，所以相邻线程写地址相差 1024 个 float。
3. 理想连续写 32 个 float 只需 1 次 128B transaction；非合并写中 32 个线程写到相隔很远的地址，可能需要约 32 次 transaction。

Exercise 5: Tiled Transpose Analysis

题目原文

Question 1: Why do we add `+1` to the shared memory declaration (`tile[BLOCK_SIZE][BLOCK_SIZE + 1]`)?

Your answer: _____

Question 2: In the write step, why do we swap `blockIdx.x` and `blockIdx.y`?

Your answer: _____

Question 3: How much shared memory does each block use for `BLOCK_SIZE=32`?

Your calculation: _____ bytes

题解

1. `+1` 改变 shared memory 每行跨度，避免转置访问时多个线程落到同一个 bank，减少 bank conflict。
2. 转置会交换 tile 的行列位置，因此写回输出矩阵时 block 的 x/y 方向也要交换。
3. 若元素为 `float`，shared memory 为 $32 * (32 + 1) * 4 = 4224$ bytes。

Challenge Exercises

题目原文

Challenge 1: Implement Tiled Transpose. Modify `main.hip` to use the shared memory tiling approach from Section 9. Compare performance with the naive version.

Challenge 2: Bank Conflict Analysis. Test with and without the `+1` padding in shared memory. Measure the performance difference.

Challenge 3: In-Place Transpose. For square matrices, implement in-place transpose using only $O(1)$ extra memory.

Challenge 4: Rectangular Tile Sizes. Experiment with non-square tile sizes (e.g., 32×8 or 16×64). When might these be beneficial?

Challenge 5: Double-Buffering. Implement double-buffering to overlap memory transfers with computation for very large matrices.

题解

这些为扩展实现题。核心思路：tiled transpose 用 shared memory 把非合并写转为更连续的读写；+1 padding 用于减少 bank conflict；in-place transpose 只适合方阵，可交换 (i, j) 和 (j, i) ；矩形 tile 在矩阵长宽差异大或希望改善 occupancy/访存时有用；double-buffering 可用 stream 和异步拷贝隐藏 PCIe/HBM 传输开销。

Lab3 Histogram

Exercise 1: Analyze the Problem

题目原文

Question 1: How many times is each input element read from global memory?

Your answer: _____

Question 2: If `num_bins = 256` and `N = 10M`, what is the total global memory traffic?

Your calculation: _____

Question 3: Why is this approach inefficient?

Your answer: _____

题解

1. 每个元素被读取 `num_bins` 次，因为每个 bin 对应一个 block 并扫描整个输入。
2. 读取次数为 $N * \text{num_bins} = 10M * 256 = 2.56B$ 。若每个输入为 4B int，则约 $2.56B * 4 = 10.24GB$ 。
3. 低效原因是重复扫描全数组，global memory traffic 极大，且每个 block 只统计一个 bin，整体工作重复严重。

Exercise 2: Understand the Optimization

题目原文

Question 1: Why is atomicAdd on shared memory faster than on global memory?

Your answer: _____

Question 2: In the merge step, how many global atomicAdd operations occur per bin?

Your answer: _____

Question 3: What is the purpose of the grid-stride loop pattern `for (int i = idx; i < N; i += totalthread)?`

Your answer: _____

题解

- 1. shared memory 位于片上，延迟低、带宽高，atomic 作用范围局限在 block 内，通常比 global memory atomic 更快。
- 2. 每个 block 对每个 bin 合并一次，所以每个 bin 的 global atomicAdd 次数等于 block 数。
- 3. grid-stride loop 让有限数量线程覆盖任意长度输入，同时保持连续线程初始访问连续元素，便于扩展到大数组。

Exercise 3: Record Results

题目原文

Fill in the execution times:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	_____	_____
Naive GPU	_____	_____
Optimized GPU	_____	_____

Calculate speedups:

- Optimized vs Serial: _____ x
- Optimized vs Naive: _____ x

题解

五次 kernel/CPU 计时的均值为：

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	2.3936 ms	23.3382 ms
Naive GPU	5.5709 ms	360.7488 ms
Optimized GPU	0.1866 ms	1.8208 ms

- Optimized vs Serial: testcase 2 为 **12.83x**，testcase 4 为 **12.82x**。
- Optimized vs Naive: testcase 2 为 **29.85x**，testcase 4 为 **198.13x**。

六组 testcase 的 optimized 输出均与 CPU 结果一致。

Exercise 4: Memory Efficiency

题目原文

Calculate the memory traffic reduction ratio:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{N \times \text{num_bins}}{N} = ?$$

Your answer: _____

题解

化简后为 **num_bins**。如果 **num_bins=256**，优化版本相对 naive 版本的读取次数理论上减少 256 倍。

Exercise 5: Contention Scenarios

题目原文

Scenario 1: All input values are the same (e.g., all zeros)

- What happens to performance? _____

- Why? _____

Scenario 2: Input values are uniformly distributed across all bins

- Expected contention level: _____
- Why? _____

题解

场景 1：性能下降，因为所有线程集中更新同一个 bin，atomic contention 最高，操作趋于串行化。场景 2：冲突较低，因为更新分散到多个 bins，atomic 竞争被摊薄。

Challenge Exercises

题目原文

Challenge 1: Implement Warp-Level Histogram. Modify the kernel to use warp-level privatization: each warp maintains its own local histogram; reduces shared memory contention further.

Challenge 2: Benchmark with Different Distributions. Modify `geninput.py` to generate uniform distribution, Gaussian distribution, highly skewed distribution. Compare performance.

Challenge 3: Vary Number of Bins. Test performance with `num_bins = {16, 64, 256, 1024}`. How does bin count affect shared memory usage? At what point does the optimization become less effective?

题解

warp-level privatization 可进一步减少同一 shared histogram 的竞争；分布越 skewed，atomic 冲突越严重；bin 数增加会增加 shared memory 占用，当 bin 数过多导致 shared memory 不足或 occupancy 降低时，局部 histogram 优化效果会下降。

Lab4 Reduction

Exercise 1: Understand Tree Reduction

题目原文

Question 1: For an array of 1024 elements with block size 256, how many reduction steps occur within each block?

Your answer: _____

Question 2: After block-level reduction, how many partial sums remain?

Your answer: _____

Question 3: Why do we use `__syncthreads()` after each reduction step?

Your answer: _____

题解

- 1. `log2(256)=8` 步。
- 2. `ceil(1024/256)=4` 个 partial sums。
- 3. 每轮 reduction 后，下一轮会读取上一轮写入的 shared memory 值，因此必须用 `__syncthreads()` 保证同一 block 内所有线程都完成当前轮。

Exercise 2: Analyze Occupancy Results

题目原文

From the test output, fill in the table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	_____	_____	_____
128	_____	_____	_____
256	_____	_____	_____
512	_____	_____	_____
1024	_____	_____	_____

Question: Does higher occupancy always mean better performance?

Your answer: _____

题解

设备上限为 2,048 threads/WGP、64 wavefronts/WGP。Tree Reduction 每 block 动态 LDS 为 $\text{block_size} \times 4 \text{ bytes}$ ；在本组 block size 下，线程数先于 64 KiB LDS 成为驻留限制，因此五种配置的资源上限 occupancy 均为 100%。执行时间采用 Tree (Shared Memory) 五次均值：

Block Size	Warps/Block	Active Blocks/WGP	Occupancy (%)	Tree Time mean $\pm$ SD (ms)
64	2	32	100	0.02234 $\pm$ 0.00023
128	4	16	100	0.02366 $\pm$ 0.00023
256	8	8	100	0.02484 $\pm$ 0.00015
512	16	4	100	0.02664 $\pm$ 0.00021
1024	32	2	100	0.02912 $\pm$ 0.00016

补充扫描同时记录了四种算法的五次统计：

Block Size	Naive Atomic (ms)	Tree (ms)	Tree + Shuffle (ms)	Multi-Element (ms)
64	0.03044 $\pm$ 0.00019	0.02234 $\pm$ 0.00023	0.01852 $\pm$ 0.00019	0.03388 $\pm$ 0.00074
128	0.05278 $\pm$ 0.04637	0.02366 $\pm$ 0.00023	0.01980 $\pm$ 0.00019	0.01976 $\pm$ 0.00043
256	0.05124 $\pm$ 0.04492	0.02484 $\pm$ 0.00015	0.02014 $\pm$ 0.00026	0.01264 $\pm$ 0.00025
512	0.04334 $\pm$ 0.02614	0.02664 $\pm$ 0.00021	0.02184 $\pm$ 0.00026	0.01030 $\pm$ 0.00029
1024	0.03368 $\pm$ 0.00061	0.02912 $\pm$ 0.00016	0.02538 $\pm$ 0.00022	0.01194 $\pm$ 0.00049

所有 25 次运行均得到 -74,993,790，并通过 **Verification: ALL PASSED**。更高 occupancy 不一定更快：五种配置的资源上限 occupancy 相同，但 Tree 时间随 block size 从 64 增至 1024 而逐步增加；Multi-Element 则在 512 threads/block 取得最低均值 0.01030 ms。算法工作分配、归约阶段的空闲线程数、atomic 数量和同步成本比单一 occupancy 数字更能解释性能。

Exercise 3: Think About Trade-offs

题目原文

Question 1: With block size 64, how many final atomic operations occur for $N = 1\text{M}$ elements?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 64) = \underline{\hspace{2cm}}$

Question 2: With block size 1024, how many final atomic operations occur?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 1024) = \underline{\hspace{2cm}}$

Question 3: Why might fewer atomic operations improve performance?

Your answer:

题解

block size 64: $\text{ceil}(1,000,000/64)=15625$ 。 block size 1024:

$\text{ceil}(1,000,000/1024)=977$ 。更少 atomic 可以减少 global memory 冲突和串行化，也减少最终合并阶段的工作量。

Exercise 4: Algorithm Analysis

题目原文

Question 1: In tree reduction, what fraction of threads are idle after the first step?

Your answer:

Question 2: Why does warp shuffle not require `__syncthreads()`?

Your answer:

Question 3: How does multi-element per thread improve memory bandwidth utilization?

Your answer:

题解

1. 第一轮后约一半线程空闲。

2. warp/wavefront 内线程天然同步执行，shuffle 在同一个 wavefront 内交换数据，不需要 block 级同步。
3. 每线程处理多个元素可增加连续访问和局部累加，减少 block 数、global atomic 数和调度开销，从而提高带宽利用率。

Challenge Exercises

题目原文

Challenge 1: Implement Tree Reduction. Modify `main.hip` to use shared memory tree reduction instead of naive atomics.

Challenge 2: Add Warp Shuffle. For the last 64 threads, use warp shuffle intrinsics: `__shfl_down()` for AMD GPUs; eliminate `__syncthreads()` for intra-warp operations.

Challenge 3: Multi-Pass Reduction. For very large arrays, implement a two-pass reduction: first pass reduce to partial sums; second pass reduce partial sums to final result.

Challenge 4: Benchmark Different Operations. Modify the kernel to compute maximum value, minimum value, product. Compare performance differences.

题解

shared memory tree reduction 可降低 global atomic 冲突；wave shuffle 可优化最后一个 wavefront 内的归约；超大数组通常需要多 pass reduction；max/min 与 sum 类似，product 需注意溢出和数值稳定性。

Lab5 Monte Carlo Integration

Exercise 1: Understand the Algorithm

题目原文

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer: _____

Question 2: What is the time complexity of the atomic reduction approach?

Your answer: _____

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer: _____

题解

1. 每个线程直接累加加权贡献，最终 atomic 的就是积分贡献；也可以改为最后统一乘 $(b-a)/n_samples$ 。
2. 逻辑工作量是 $O(n_samples)$ ，但 atomic 冲突会增加实际开销。
3. 每个 block 先在 shared memory 中归约局部和，再由每个 block 写出一个 partial 或做一次 global atomic。

Exercise 2: Manual Calculation

题目原文

For Test 1, verify the result manually:

Given: $a = 0$, $b = 2$, $n = 8$ samples

Formula: $result = \frac{(b-a)}{n} \sum_{i=1}^n y_i = \frac{2}{8} \times \sum y_i$

Your calculation:

- Sum of y values: _____
- Expected result: _____
- Does it match the program output? _____

题解

程序输出的 8 个采样值为 4.1805, 1.7864, 4.7554, 3.9983, 2.2786, 1.1873, 4.1222, 1.4420。

- $sum(y_i) = 23.7507$
- $Expected\ result = 0.25 \times 23.7507 = 5.937675$
- 程序输出为 5.937675，完全匹配。

Exercise 3: Scalability Analysis

题目原文

Record the execution times:

Test	Sample Count	Execution Time
4	100,000	_____
7	10,000,000	_____
8	100,000,000	_____

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer: _____

Question 2: Is the scaling linear? Why or why not?

Your answer: _____

题解

实测 shell wall time 为：100,000 样本= 0.072 s，10,000,000 样本= 0.894 s，100,000,000 样本= 8.454 s。

从 100K 增长到 10M，样本数增加 $100\times$ ，时间增加 $0.894/0.072=12.42\times$ ；小规模固定启动、输入和进程开销使该区间不呈线性。从 10M 增长到 100M，样本数增加 $10\times$ ，时间增加 $8.454/0.894=9.46\times$ ，大规模区间已接近线性。

Exercise 4: Bottleneck Analysis

题目原文

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for $N=1M$ threads?

Hint: Total work = $N \times (\text{compute} + \text{atomic})$

Your calculation: _____

Question 2: How would shared memory reduction improve this?

Your answer: _____

Question 3: With 256 threads per block and $N=1M$, how many global atomics would shared memory reduction require?

Your calculation: _____

题解

1. 理论效率约为 $1 / (1 + 100) = 0.99\%$ ，atomic 占主要开销。
2. shared memory reduction 先在 block 内合并，把大量 global atomic 降为每个 block 一次。
3. 若每 256 个样本一个 block，则 global atomic 数约为 $\text{ceil}(1,000,000/256)=3907$ 。若实现固定使用 256 blocks，则 global atomic 为 256。

Exercise 5: Precision Considerations

题目原文

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer: _____

Question 2: If we switch to `float` (32-bit), what problems might occur with $N=100M$ samples?

Your answer: _____

题解

`double` 通常提供约 15 到 16 位十进制有效数字。`float` 只有约 6 到 7 位有效数字， $N=100M$ 时大量小量累加容易出现舍入误差、低位丢失，最终积分精度下降。

Exercise 6: Optimization Analysis

题目原文

Question 1: In the optimized version, how many global atomics occur for N=1M with 256 blocks?

Your answer: _____

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x)?`

Your answer: _____

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer: _____

题解

1. 256 blocks 时，每个 block 一个 global atomic，共 256 次。
2. grid-stride loop 让固定数量线程覆盖任意样本数，并让每个线程处理多个样本。
3. 缩放因子对所有样本相同，只需最终统一乘一次，避免每个线程重复计算同一个乘法。

Challenge Exercises

题目原文

Challenge 1: Implement Shared Memory Reduction. Modify `main.hip` to use the two-phase approach shown in Section 9. Compare performance.

Challenge 2: Compute Pi Using Monte Carlo. Estimate π by sampling points in a square and counting how many fall inside a quarter circle: $\pi \approx 4 \times \frac{\text{points inside circle}}{\text{total points}}$.

Challenge 3: Error Analysis. For a known integral, run with N = 1000, 10000, 100000, 1000000; calculate absolute error; verify the $1/\sqrt{n}$ convergence rate.

Challenge 4: Compare float vs double. Modify the kernel to use `float` instead of `double`; compare accuracy and execution time.

题解

共享内存两阶段归约可减少 global atomic；估计 Pi 时每个线程独立采样点并统计命中；Monte Carlo 误差通常按 $1/\text{sqrt}(n)$ 收敛；float 更快但精度更低，double 更稳但可能更慢。

Lab6 K-Means Clustering

Exercise 1: Algorithm Analysis

题目原文

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer: _____

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer: _____

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer: _____

题解

1. 每个线程处理一个点，需要比较 100 个中心，复杂度为 $O(100)$ ，一般写作 $O(k)$ 。
2. 先用 shared memory atomic 在 block 内局部累加，可减少昂贵的 global atomic 冲突。
3. 空 cluster 无法用新点均值更新，通常保持上一轮中心不变，或按策略重新初始化。

Exercise 2: Performance Analysis

题目原文

Record the execution time for test 4:

- Sample size: 50,000
- Clusters (k): 50
- Max iterations: 100
- Execution time: _____

Question: What is the dominant computation in each iteration?

Your answer: _____

题解

test 4 的实测 shell wall time 为 **0.081 s** (**real 0m0.081s**)。每轮 dominant computation 是 assignment 阶段的距离计算，即 $N \times k = 50,000 \times 50 = 2,500,000$ 次点到中心距离计算。

Exercise 3: Compute Intensity

题目原文

For $N = 50,000$ points and $k = 50$ clusters:

Assignment kernel:

- Distance calculations per point: _____
- Total distance calculations: _____

Update kernel:

- Shared memory atomics: _____
- Global atomics (with 256 threads/block): _____

题解

assignment: 每点 50 次距离计算，总计 $50,000 * 50 = 2,500,000$ 次。update: 若每点对 **sum_x**、**sum_y**、**count** 做 3 次 shared atomic，则 shared atomic 为 $50,000 * 3 = 150,000$ 。256 threads/block 时 $\text{blocks} = \text{ceil}(50000/256) = 196$ ；若每 block 对每 cluster 写 3 个全局统计量，则 global atomic 为 $196 * 50 * 3 = 29,400$ 。

Exercise 4: Memory Optimization

题目原文

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer: _____

Question 2: If $k = 1000$, what would be the impact on cache performance?

Your answer: _____

题解

1. 可将 centroids 放入 constant memory、shared memory tile，或用 SoA 布局增强 coalescing/cache 命中。
2. $k=1000$ 时中心数组更大，每个线程读取更多中心，cache 压力增大，可能导致更多 cache miss 和更高 memory traffic。

Exercise 5: Convergence Analysis

题目原文

Question 1: What is the convergence threshold in this implementation?

Your answer: _____

Question 2: Why is it important to check convergence rather than always running max_iterations?

Your answer: _____

Question 3: How does the double-buffering (swapping prev and new centroids) help?

Your answer: _____

题解

1. 阈值为 $1.0e-8f$ 。
2. 若中心已收敛，提前停止可以避免无效迭代，减少计算时间。
3. double-buffering 让一轮迭代读旧中心、写新中心，轮末交换指针，避免读写同一数组造成数据混乱。

Challenge Exercises

题目原文

Challenge 1: Support Larger k. Modify the kernel to use dynamic shared memory allocation:

```
extern __shared__ float shared_mem[];
```

Challenge 2: Add Distance Output. Modify the program to output the sum of squared distances (inertia) after clustering.

Challenge 3: Extend to 3D. Add support for 3D point clustering by adding z coordinates.

Challenge 4: K-Means++ Initialization. Implement K-Means++ initialization on GPU: choose first centroid randomly; for each subsequent centroid, choose with probability proportional to distance squared.

Challenge 5: Mini-Batch K-Means. Process random subsets of points each iteration to speed up convergence on very large datasets.

题解

动态 shared memory 支持更大的 k，但要注意 LDS 容量和 occupancy；inertia 可在 assignment 阶段累加最近距离平方；3D 扩展需要增加 z 坐标和三维距离；K-Means++ 可改善初始中心质量但并行实现更复杂；Mini-Batch K-Means 每轮只处理子集，适合大数据但结果有随机性。